

Lab 4: VibeCoding

Match-3 Gem Swap (Problem Statement 57) — Full Chat History

All prompts and Claude's responses, in order, reproduced in full

USER PROMPT — Prompt 1 (Task 1)

[Attached files: Lab_4_VibeCoding_Student_handout.pdf and VibeCodingGitRepoLinks.pdf - full text reproduced in the Appendix at the end of this document]

my problem statement is 57 , I am working on the Match-3 Gem Swap game for a Vibe Coding lab.

Please implement Task 1 from the README only.

The bug is in game/board.py inside Board.process_swap().

Currently, when two adjacent gems are swapped and the swap does NOT create a match, the gems are correctly swapped back, but `self.moves_remaining` is still decreased by 1.

Required behavior:

1. A valid adjacent swap that creates a match should decrease `moves_remaining` by exactly 1.
2. An adjacent swap that creates no match should be reverted and should NOT decrease `moves_remaining`.
3. Non-adjacent swaps, game-over states, and animations should continue to behave as before.
4. Do not modify cascade scoring, Bomb Gems, idle hints, rendering, or any unrelated functionality.
5. Make the smallest safe code change possible.

First explain the current bug and the proposed fix. Then provide only the modified `process_swap()` function. and give me the full upadted code ?

CLAUDE RESPONSE — Response 1 (Task 1)

I'll pull up the repo first so the fix is based on your actual code.

[Claude fetched the repository page https://github.com/SETAPESU26/57_match_three_gem and read the README. A second fetch of the game/ folder was refused by GitHub ("Site disallows automated access"), so `board.py` could not be read.]

I read the README, but GitHub blocked me from opening `game/board.py`, so I can't see your actual code. I won't invent a full file that might not match yours. Paste `board.py` here and I'll give you the complete updated file with the smallest change applied.

The current bug

The README says `self.moves_remaining -= 1` runs before the game checks whether the swap made a match. So every adjacent swap costs a move, even when it gets reverted.

The proposed fix

1. Delete the early `self.moves_remaining -= 1`.

2. Put it inside the branch where matches were found, just before matches are resolved.
3. Leave the no-match branch (swap back) untouched, so it never touches the counter.

`process_swap()` shape (adapt to your variable names)

I'm guessing at your names (`find_matches`, `resolve_matches`, `swap_gems`, `selected`, `game_over`), so treat this as a template:

```
def process_swap(self, r1, c1, r2, c2):
    # Non-adjacent swaps and game-over handling: keep exactly as before
    if self.game_over or not self.is_adjacent(r1, c1, r2, c2):
        return False

    # (REMOVED from here) self.moves_remaining -= 1

    self.swap_gems(r1, c1, r2, c2)
    matches = self.find_matches()

    if matches:
        self.moves_remaining -= 1      # moved here: only valid swaps cost a move
        self.resolve_matches()        # unchanged
        return True
    else:
        self.swap_gems(r1, c1, r2, c2) # revert, no move deducted
        return False
```

Your three-line change

- Remove the `self.moves_remaining -= 1` line that sits above the match check.
- Add `self.moves_remaining -= 1` inside the `if matches:` branch.
- Change nothing else.

One check when you paste your code

If your swap uses animations or a state machine (for example, a "swapping" state that reverts after an animation finishes), the match check may happen in a different function than `process_swap()`. The deduction would then need to go where the match result is known. Pasting the file lets me place it correctly.

After the fix

1. Make a swap that creates a match. Moves should drop by 1.
2. Make a swap that creates no match. The gems should revert and moves should stay the same.

Record the "before" video on the unmodified code first, since the lab needs it.

USER PROMPT — Prompt 2 (Task 2)

Task 2: Implement cascade combo scoring in the Python Pygame Match-3 game.

The current `Board.resolve_matches()` repeatedly finds matches, clears them, drops/refills the board, and continues until no matches remain.

I need cascade scoring with these rules:

1. The first match in a swap uses a 1x multiplier.
2. If the drop/refill automatically creates another match, that is the second cascade and uses a 2x multiplier.
3. The third cascade uses 3x, and so on.
4. Score for each cascade should be:
 $\text{number_of_gems_cleared} \times 10 \times \text{cascade_multiplier}$
5. The cascade multiplier must reset when the complete chain ends.
6. A normal swap that produces only one match should therefore use 1x.
7. Keep the existing Task 1 behavior: an invalid swap must not consume a move.
8. Do not implement Bomb Gems or idle hints yet.
9. Do not change rendering or unrelated game behavior.
10. Make the smallest safe change possible.

First explain how the current `resolve_matches()` and `process_swap()` work and where the cascade multiplier should be applied. Then provide the modified code.

CLAUDE RESPONSE — Response 2 (Task 2)

I still can't see your `board.py`, so I can't describe how your `resolve_matches()` and `process_swap()` work line by line. GitHub blocks automated access to the `game/` folder. Below is the change based on the README's description (find matches, clear, drop and refill, repeat), with names you'll need to match to yours. If you paste `board.py`, I'll return the exact modified file.

How it works now (per the README)

- `process_swap()` swaps two gems and checks for a match. With your Task 1 fix, it deducts a move only if a match exists. It then calls `resolve_matches()`.
- `resolve_matches()` loops: find matches, clear them, add a flat $10 \times \text{gems}$ to the score, drop and refill, and repeat until `find_matches()` returns nothing.

Where the multiplier goes

Each pass of that loop is one cascade, so the multiplier belongs in `resolve_matches()`. Keep a local counter that starts at 1 and increments after each pass. Because it's a local variable, it resets automatically every time the function is called. That means it resets when a chain ends, with no state to clean up. `process_swap()` needs no change, so Task 1 behavior stays intact.

Modified `resolve_matches()` (template)

```
def resolve_matches(self):
    cascade = 1                                # NEW: 1x for first match, resets each call
    matches = self.find_matches()
    while matches:
        cleared = self.clear_matches(matches)    # however you currently clear; need the count
        self.score += len(cleared) * 10 * cascade # CHANGED: was len(cleared) * 10
        self.apply_gravity_and_refill()          # unchanged
        cascade += 1                             # NEW: next pass is 2x, 3x, ...
        matches = self.find_matches()
```

The three edits

1. Add `cascade = 1` before the loop.
2. Change the scoring line to `gems_cleared * 10 * cascade`. Make sure `gems_cleared` counts unique gems, not gems counted once per line, if an L or T shape can overlap.
3. Add `cascade += 1` at the end of each loop pass.

Things to check against your real code

- If matches are stored as a set of `(row, col)` coordinates, `len(matches)` is the unique gem count. If they're stored as a list of runs, sum the set of positions instead.
- If your drop and refill is animated through a state machine (e.g. `resolve_matches()` only runs one step per frame), a local variable won't persist. You'd then need a `self.cascade_level` attribute, reset to 1 when a new swap starts and incremented after each step. Paste the file and I'll tell you which case you have.

Test it

1. A simple 3-match should score 30.
2. A swap that triggers one automatic follow-up match should score `n×10` for the first clear plus `m×10×2` for the second.
3. After the chain ends, your next swap should start at 1x again.
4. An invalid swap should still cost no move.

USER PROMPT — Prompt 3 (Task 3)

I am working on the Match-3 Gem Swap game for a Vibe Coding lab.

Implement Task 3 from the README only: Bomb Gem.

Requirements:

1. When a horizontal or vertical match of 4 gems of the same color is created, create a special Bomb Gem.
2. The Bomb Gem must be visually distinguishable from normal gems using the existing Pygame rendering system. Do not require external image assets.
3. The Bomb Gem should retain its original gem color.
4. When an existing Bomb Gem is activated as part of a match, it should clear its entire row OR entire column.
5. For a horizontal 4-match, the Bomb Gem may be configured to clear its row.
6. For a vertical 4-match, the Bomb Gem may be configured to clear its column.
7. Normal 3-gem matches must continue to work exactly as before.
8. Task 1 behavior must remain unchanged: an invalid swap must not consume a move.
9. Task 2 cascade scoring must remain unchanged:
 - first cascade = 1x
 - second cascade = 2x
 - third cascade = 3x
 - etc.
 - score = gems cleared × 10 × cascade multiplier
10. Do not implement the idle hint feature yet.
11. Do not change unrelated game behavior.
12. Keep the implementation as small and safe as possible.
13. Do not introduce external dependencies or image files.

Before changing the code:

- Explain how the current Gem and Board classes work.
- Explain how 4-in-a-row detection will be added.
- Explain how a Bomb Gem will be represented.
- Explain how Bomb activation will clear a row or column.
- Explain how the existing cascade scoring will continue to work.

Then provide the modified code needed for Task 3.

CLAUDE RESPONSE — Response 3 (Task 3)

I still can't open `game/board.py` (GitHub blocks automated access to that folder), so I can't describe your `Gem` and `Board` classes as they really are. Below I explain the design based on the README's description, then give template code using assumed names. Paste `board.py` and I'll adapt it into an exact, drop-in version.

Assumptions about your current code

- **Gem:** a gem has a color (an int, string or `Gem` object).

- **Board:** it holds an 8x8 grid `self.grid[r][c]` and has `find_matches()`, `resolve_matches()` (Task 2's cascade loop), gravity/refill, and `process_swap()` (Task 1).
- **Rendering:** a draw method loops over the grid and draws each gem with Pygame shapes, e.g. `pygame.draw.circle`.

How the pieces will work

4-in-a-row detection. Keep `find_matches()` as it is, so 3-matches behave exactly as before. Add one small helper, `find_four_runs()`, that scans rows and columns for runs of 4 or more same-colored gems and returns each run with its orientation. It is called before clearing, inside the same cascade pass.

Bomb representation. Add two attributes to the gem: `is_bomb` (default `False`) and `bomb_dir` ("row" or "col"). The color is untouched, so the bomb keeps its original color. If your grid stores plain ints or strings instead of `Gem` objects, tell me and I'll use a parallel `self.bombs` dict instead, which must move with gravity.

Spawning. When a 4-run is cleared, one cell of the run is kept and turned into a bomb instead of being removed. A horizontal run gives `bomb_dir="row"`, and a vertical run gives "col". The bomb is placed at one of the swapped cells if it's in the run, otherwise at the run's second cell.

Activation. When a bomb is part of the cells being cleared, its whole row or column is added to the clear set. Bombs caught in that blast also go off, so chains work. A newly spawned bomb is never destroyed by its own creation.

Cascade scoring. Scoring is unchanged: `len(cleared) × 10 × cascade`. The bomb blast just adds more cells to that pass's `cleared` set, so they all score at the current multiplier. The local `cascade` counter and its reset are not touched, and `process_swap()` is not touched either, so Task 1 stays intact.

Code

1. Gem

```
class Gem:
    def __init__(self, color):
        self.color = color
        self.is_bomb = False          # NEW
        self.bomb_dir = None          # NEW: "row" or "col"
```

2. New helper in Board

```
def find_four_runs(self):
    """Return [(cells, 'row'|'col'), ...] for runs of 4+ same-colored gems."""
    runs = []
    n = len(self.grid)
    for r in range(n):
        # horizontal
        c = 0
        while c < n:
            e = c
            while (e + 1 < n and self.grid[r][c] and self.grid[r][e + 1]
                  and self.grid[r][e + 1].color == self.grid[r][c].color):
                e += 1
            if e - c + 1 >= 4:
                runs.append([(r, k) for k in range(c, e + 1)], "row")
            c = e + 1
        for c in range(n):
            # vertical
            r = 0
            while r < n:
                e = r
                while (e + 1 < n and self.grid[r][c] and self.grid[e + 1][c]
                      and self.grid[e + 1][c].color == self.grid[r][c].color):
                    e += 1
                if e - r + 1 >= 4:
                    runs.append([(k, c) for k in range(r, e + 1)], "col")
                r = e + 1
    return runs
```

3. Modified `resolve_matches()`

Only the marked lines are new or changed. Replace `clear_cells` and `apply_gravity_and_refill` with whatever your code uses.

```

def resolve_matches(self):
    cascade = 1
    matches = self.find_matches()          # set of (r, c), unchanged
    while matches:
        runs4 = self.find_four_runs()      # NEW
        to_clear = set(matches)

        # NEW: expand bombs (row/col clear, chain reactions)
        queue = [p for p in to_clear if self.grid[p[0]][p[1]].is_bomb]
        blast = set()
        while queue:
            r, c = queue.pop()
            gem = self.grid[r][c]
            n = len(self.grid)
            cells = ([ (r, k) for k in range(n)] if gem.bomb_dir == "row"
                     else [(k, c) for k in range(n)])
            for p in cells:
                if p not in to_clear:
                    to_clear.add(p)
                    blast.add(p)
                    g = self.grid[p[0]][p[1]]
                    if g and g.is_bomb:
                        queue.append(p)

        # NEW: choose which cells become bombs (not cleared)
        spawn = {}
        for cells, orient in runs4:
            pos = cells[1]
            for p in cells:
                if p in getattr(self, "last_swap", ()):
                    pos = p
                    break
            if pos not in blast:
                spawn[pos] = orient
        for pos in spawn:
            to_clear.discard(pos)

        self.score += len(to_clear) * 10 * cascade    # unchanged formula
        self.clear_cells(to_clear)                  # your existing clear code

        for (r, c), orient in spawn.items():        # NEW
            self.grid[r][c].is_bomb = True
            self.grid[r][c].bomb_dir = orient

        self.apply_gravity_and_refill()              # unchanged
        cascade += 1                                # unchanged
        matches = self.find_matches()

```

For the swap-position preference, set `self.last_swap = {(r1, c1), (r2, c2)}` inside `process_swap()` right after the swap. It's a single assignment and doesn't affect Task 1. If you skip it, the bomb simply spawns at `cells[1]`.

4. Rendering (in your draw code, after drawing each gem)

```

if gem.is_bomb:
    cx, cy = rect.centerx, rect.centery          # rect = the gem's tile rect
    pygame.draw.circle(screen, (255, 255, 255), (cx, cy), rect.width // 2 - 4, 3)
    if gem.bomb_dir == "row":
        pygame.draw.line(screen, (255, 255, 255), (rect.left + 8, cy), (rect.right - 8, cy), 3)
    else:
        pygame.draw.line(screen, (255, 255, 255), (cx, rect.top + 8), (cx, rect.bottom - 8), 3)

```

The bomb keeps its gem color and gets a white ring plus a line showing whether it clears a row or a column.

Things to verify against your real code

- If refilled or dropped gems are created as new `Gem` objects, make sure gravity moves the existing object (so `is_bomb` travels with it) rather than copying only the color.
- If `find_matches()` returns runs instead of a set of positions, the `to_clear = set(matches)` line needs a small adjustment.
- If animations run as a state machine, the cascade logic may span frames. Paste the file and I'll handle that case.

Test checklist

1. A plain 3-match still scores 30 and creates no bomb.
2. A horizontal 4-match leaves a row bomb, and a vertical 4-match leaves a column bomb, in the original color.
3. Matching that bomb clears its whole row or column.
4. Cascade multipliers still go 1x, 2x, 3x.
5. A swap with no match still costs no move.

USER PROMPT — Prompt 4 (Task 4)

Task 4: Implement the idle hint feature in the Python Pygame Match-3 game.

The current game already contains:

1. Task 1: Invalid swaps do not consume a move.
2. Task 2: Cascade combo scoring:
 - first cascade = 1x
 - second cascade = 2x
 - third cascade = 3x
3. Task 3: Bomb Gems created by 4-in-a-row matches, with row/column clearing.

Implement ONLY Task 4 from the README.

Requirements:

1. If the player has not made an interaction/move for 5 seconds, the game should automatically find a possible valid adjacent gem swap that would create a match.
2. Highlight the two gems involved in the suggested swap.
3. The hint should be visually noticeable, preferably using a pulsing outline.
4. The hint should disappear when the player makes a swap attempt or when the board changes.
5. Reset the 5-second idle timer after player interaction.
6. If there is no valid move available, do not display a hint.
7. Do not change the existing matching rules.
8. Do not change Task 1 invalid-swap behavior.
9. Do not change Task 2 cascade scoring.
10. Do not change Task 3 Bomb Gem behavior.
11. Do not add external dependencies or image assets.
12. Keep the implementation small and safe.
13. Use the existing Pygame rendering system.

The current Board class has:

- find_matches()
- process_swap()
- update()
- render()
- swap_gems()

First explain where the idle timer, hint search, hint reset, and hint rendering should be added.

Then provide the complete updated game/board.py file.

CLAUDE RESPONSE — Response 4 (Task 4)

I still can't open `game/board.py` (GitHub blocks the `game/` folder), so I can't honestly hand you a complete updated file. Writing one blind would overwrite your Task 1-3 work with guesses about names and layout. Below is the design and the exact additions. Paste your current `board.py` and I'll return the full file with these merged in.

Where each piece goes

- **Idle timer:** one attribute, `self.last_interaction`, set in `__init__`. It is checked in `update()`, which runs every frame.
- **Hint search:** a new method, `find_hint()`. It tries each adjacent pair, swaps the two cells in the grid, calls your existing `find_matches()`, and swaps back. No matching rules change, and it never calls `process_swap()`, so Task 1 and the move counter are untouched.
- **Hint reset:** a small `clear_hint()` method. It is called at the very start of `process_swap()` (any swap attempt), at the end of `resolve_matches()` (board changed), and when the player clicks a gem. It also restarts the idle timer.
- **Hint rendering:** a few lines at the end of `render()`. It draws a pulsing outline around the two hinted cells, using `math.sin` and `pygame.draw.rect`.

Code to add

1. Imports (top of file, if not already there)

```
import math
import pygame
```

2. In `Board.__init__`

```
self.hint = None                # NEW: ((r1, c1), (r2, c2)) or None
self.last_interaction = pygame.time.get_ticks() # NEW
self.HINT_DELAY_MS = 5000      # NEW
```

3. New methods

```
def reset_idle(self):
    self.hint = None
    self.last_interaction = pygame.time.get_ticks()

def find_hint(self):
    n = len(self.grid)
    for r in range(n):
        for c in range(n):
            for dr, dc in ((0, 1), (1, 0)): # right and down cover every pair once
                r2, c2 = r + dr, c + dc
                if r2 >= n or c2 >= n:
                    continue
                g = self.grid
                g[r][c], g[r2][c2] = g[r2][c2], g[r][c] # try the swap
                found = bool(self.find_matches())
                g[r][c], g[r2][c2] = g[r2][c2], g[r][c] # always swap back
                if found:
                    return ((r, c), (r2, c2))
    return None # no valid move: no hint
```

I swap grid cells directly instead of calling `swap_gems()`, in case that method triggers animation or sound. If your `swap_gems()` is a plain swap, either way works.

4. In `update()` (add at the end, guarded so it doesn't fire during animations or game over)

```
busy = getattr(self, "animating", False) # use your real animation flag
if (not busy and not self.game_over and self.hint is None
    and pygame.time.get_ticks() - self.last_interaction > self.HINT_DELAY_MS):
    self.hint = self.find_hint()
    if self.hint is None: # no moves: don't search every frame
        self.last_interaction = pygame.time.get_ticks()
```

5. Reset points

- First line of `process_swap()`: `self.reset_idle()`

- Last line of `resolve_matches()`: `self.reset_idle()`
- Where a gem click is handled (in `Board` or `game_engine.py`): `board.reset_idle()`

6. At the end of `render()`

```
if self.hint:
    pulse = (math.sin(pygame.time.get_ticks() / 150) + 1) / 2    # 0..1
    width = 2 + int(pulse * 4)
    color = (255, 255, 120)
    for (r, c) in self.hint:
        rect = pygame.Rect(self.offset_x + c * self.cell_size,
                           self.offset_y + r * self.cell_size,
                           self.cell_size, self.cell_size)      # use your real tile rect
    pygame.draw.rect(screen, color, rect, width, border_radius=8)
```

Replace `offset_x`, `offset_y`, `cell_size` and `screen` with whatever your existing draw code uses. Copy the tile-rect calculation from there.

Things to check against your code

- **Timer units:** if your code uses `time.time()` or `dt` instead of `pygame.time.get_ticks()`, tell me and I'll switch it.
- **`find_matches()` side effects:** if it modifies state, such as marking gems, `find_hint()` would need a different check. The template assumes it is read-only.
- **Animation flag:** use whatever your code already has to block input during drops.
- **`self.game_over`:** use your actual game-over flag name.

Test checklist

1. Wait 5 seconds and two gems should pulse.
2. Click or swap and the hint should vanish and the timer restart.
3. After a cascade the hint should stay hidden until 5 more idle seconds pass.
4. On a board with no valid moves, no hint should appear.
5. Tasks 1-3 should behave as before: invalid swaps cost no move, cascades score 1x, 2x, 3x, and 4-matches create bombs.

USER PROMPT — Prompt 5 (Export request)

give me all the Prompts , and your response in the PDF , do not skip anything

CLAUDE RESPONSE — Export request

Response 5: this PDF document.

Appendix: Text of files attached to Prompt 1

A. Lab_4_VibeCoding_Student_handout.pdf

Lab 4: VibeCoding

Objective

Use Vibe Coding tools to fix the broken code and add features to the same. Learn how to prompt to get the job done in under three to four attempts. Clone a repo assigned to you, make changes and push to your repo in your personal account.

THIS IS AN INDIVIDUAL ASSIGNMENT. WORK INDIVIDUALLY!!

Duration

60 minutes

Software Requirements

- Access to Vibe Coding tool. (ChatGPT, Gemini, Claude etc)

Instructions:

1. Go to the repo assigned to you
2. Go through the ReadMe file to understand the deliverables
3. Clone or Fork the repo
4. Run the Python based code (A game/ A Puzzle)
5. Record 10 seconds of video before making any changes.
6. Write prompts and fix the broken code
7. Write prompts to add all the features listed in the ReadMe file of the repo one by one.
8. Record another 10 seconds of video again after making all the changes.
9. Push the code to your repo.
10. Do not raise PR to the main repo (SETAPESU26)

Deliverables:

Upload the following to your Lab repo under Lab-4 folder,

- a. Videos (before and after)
- b. Updated Code
- c. Chat history exported as a doc/pdf

B. VibeCodingGitRepoLinks.pdf

VibeCodingRepos

Sl. No Git Repo Link

- 1 https://github.com/SETAPESU26/01_frogger
- 2 https://github.com/SETAPESU26/02_air_hockey
- 3 https://github.com/SETAPESU26/03_fishing
- 4 https://github.com/SETAPESU26/04_target_shooting
- 5 https://github.com/SETAPESU26/05_balloon_pop
- 6 https://github.com/SETAPESU26/06_coin_collector
- 7 https://github.com/SETAPESU26/07_catch_falling
- 8 https://github.com/SETAPESU26/08_helicopter
- 9 https://github.com/SETAPESU26/09_tic_tac_toe
- 10 https://github.com/SETAPESU26/10_breakout
- 11 https://github.com/SETAPESU26/11_maze_chase
- 12 https://github.com/SETAPESU26/12_maze_runner
- 13 https://github.com/SETAPESU26/13_minesweeper
- 14 https://github.com/SETAPESU26/14_2048
- 15 https://github.com/SETAPESU26/15_connect_four
- 16 https://github.com/SETAPESU26/16_hangman
- 17 https://github.com/SETAPESU26/17_wordle
- 18 https://github.com/SETAPESU26/18_blackjack
- 19 https://github.com/SETAPESU26/19_battleship
- 20 https://github.com/SETAPESU26/20_mastermind
- 21 https://github.com/SETAPESU26/21_checkers
- 22 https://github.com/SETAPESU26/22_sliding_puzzle
- 23 https://github.com/SETAPESU26/23_dots_and_boxes
- 24 https://github.com/SETAPESU26/24_treasure_hunt
- 25 https://github.com/SETAPESU26/25_zombie_escape
- 26 https://github.com/SETAPESU26/26_bubbleBobble
- 27 https://github.com/SETAPESU26/27_centipede
- 28 https://github.com/SETAPESU26/28_defender
- 29 https://github.com/SETAPESU26/29_digDug
- 30 https://github.com/SETAPESU26/30_donkeykong
- 31 https://github.com/SETAPESU26/31_galaga
- 32 https://github.com/SETAPESU26/32_lunarLander
- 33 https://github.com/SETAPESU26/33_missileCommand
- 34 https://github.com/SETAPESU26/34_pacman

35 https://github.com/SETAPESU26/35_qbert
36 https://github.com/SETAPESU26/36_dinoRun
37 https://github.com/SETAPESU26/37_climber
38 https://github.com/SETAPESU26/39_endless-runner
39 https://github.com/SETAPESU26/39_endless-runner
40 https://github.com/SETAPESU26/40_flappy-bird
41 https://github.com/SETAPESU26/41_fruit-ninja
42 https://github.com/SETAPESU26/42_marble-tilt-maze
43 https://github.com/SETAPESU26/43_reaction-time-tester
44 https://github.com/SETAPESU26/44_simple-platformer
45 https://github.com/SETAPESU26/45_snake
46 https://github.com/SETAPESU26/46_space-invaders
47 https://github.com/SETAPESU26/47_target-aim-trainer
48 https://github.com/SETAPESU26/48_whack-a-mole
49 https://github.com/SETAPESU26/49_brick-breaker
50 https://github.com/SETAPESU26/50_traffic-escape
51 https://github.com/SETAPESU26/51_rhythm-tap
52 https://github.com/SETAPESU26/52_lava-escape
53 https://github.com/SETAPESU26/53_arm_wrestle
54 https://github.com/SETAPESU26/54_skyscraper_stack
55 https://github.com/SETAPESU26/55_card_predictor
56 https://github.com/SETAPESU26/56_tug_of_war
57 https://github.com/SETAPESU26/57_match_three_gem
58 https://github.com/SETAPESU26/58_color_pattern
59 https://github.com/SETAPESU26/59_rock_paper_scissor
60 https://github.com/SETAPESU26/60_number_guess
61 https://github.com/SETAPESU26/61_math_flashcards
62 https://github.com/SETAPESU26/62_scramble
63 https://github.com/SETAPESU26/63_anvil_dodge
64 https://github.com/SETAPESU26/64_fruit_catcher